

VERIFICATION PLAN

AXI-Lite Slave IP with Memory

ECE-593: Fundamentals of Pre-Si Validation

Portland State University

Team: Patrick, Pratibha

Document Information

Field	Value
Document ID	VPL-AXIL-001
Project / Product	AXI-Lite Slave IP with Memory (axi_lite_mem)
Design Version	RTL v0.1 (MS1 baseline, unchanged through MS5)
Verification Level	Block-level
Prepared By	Patrick, Pratibha (Verification — shared ownership)
Reviewed By	Patrick, Pratibha
Approved By	Instructor — Venkatesh Patil
Date Created	April 20, 2026
Last Revised	May 28, 2026
Revision	1.3 – MS5 Final Submission
Status	MS5 Delivered; Final Submission
Confidentiality	Internal (coursework)

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of the AXI-Lite Slave IP with Memory. It is structured in accordance with

the hierarchical verification methodology described in Comprehensive Functional Verification by Bruce Wilde.

For ECE-593: Fundamentals of Pre-Si Validation, instructed by Venkatesh Patil.

Git Repository

https://github.com/a215364731/ece593_final_project

All deliverables for each milestone live under the corresponding subdirectory:

AXI_Lite_MS1_Trad/	AXI_Lite_MS2_Class/	AXI_Lite_MS3_Class/
AXI_Lite_MS4_UVM/	AXI_Lite_MS5_UVM/	AXI_Lite_FINAL/

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying the AXI-Lite Slave IP with Memory. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan.

This document applies to the following verification level:

- ☒ Block-level verification
- ☐ Sub-system verification
- ☐ Full-chip / Top-level verification
- ☐ System-level verification

1.2 Design Overview

The Design Under Verification (DUV) is a parameterized AXI4-Lite slave that wraps an internal dual-port block RAM. The top-level module `s_axil_top` instantiates two submodules — `s_axil_wr` (handling AW, W, and B channels) and `s_axil_rd` (handling AR and R channels) — and a shared memory array of `MEM_DEPTH` words, each `DATA_WIDTH` bits wide. All five AXI4-Lite channels follow the standard VALID/READY handshake and the design supports byte-enable writes via `WSTRB`, out-of-range `SLVERR` responses, and write-before-read forwarding when a read and write target the same word in the same cycle.

Refer to HLDS-AXIL-001 (Design Specification) for the complete block diagram, port list, parameter definitions, and channel behavior.

1.3 Verification Hierarchy

Verification of this design follows a hierarchical approach consistent with Bruce Wilde's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions and feeds results to the next level.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
Block	<code>s_axil_top</code> (with <code>wr</code> , <code>rd</code> , memory)	☒ Yes	—
Sub-system	AXI interconnect + slaves	☐ No	Future VP
Full-chip	SoC integration	☐ No	Future VP
System	Multi-processor system (ECE-593 FP)	☐ No	VP-SYS-001

1.4 References

Ref #	Document Title	Version	Date
[R01]	HLDS-AXIL-001 – AXI-Lite Slave IP with Memory Specification	1.0	2026-04-20

Ref #	Document Title	Version	Date
[R02]	Comprehensive Functional Verification – Bruce Wilde	—	—
[R03]	AMBA AXI and ACE Protocol Specification (ARM IHI 0022E)	E	—
[R04]	ECE-593 Final Project Handout – V. Patil	—	2026
[R05]	SystemVerilog IEEE 1800-2017 LRM	2017	2017

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DUV	Design Under Verification
DUT	Device Under Test
UVM	Universal Verification Methodology
SV	SystemVerilog
TB	Testbench
FC	Functional Coverage
CC	Code Coverage
CR	Coverage Report
VP	Verification Plan
AXI	Advanced eXtensible Interface
WSTRB	Write byte-lane strobe signal
BRESP / RRESP	Write / Read response (2-bit AXI encoding)
SLVERR	Slave error response (RESP = 2'b10)
MS	Milestone (MS1 through MS5 + FINAL)

2. Functions to Be Verified

This section lists all functions and features of the DUV that are within scope of this verification plan, as required by the verification hierarchy (Wilde, Chapters 3–4). Each function is assigned a unique identifier for traceability.

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-001	Reset behavior — all control flags and pipeline registers initialize to defined values on resetn assertion	DS §5 Reset	Critical	Directed + FC
FV-002	AW channel VALID/READY handshake — AWREADY gating by aw_active	DS §7 AW	Critical	Directed + FC
FV-003	W channel VALID/READY handshake — WREADY gating by w_active	DS §7 W	Critical	Directed + FC
FV-004	Write commit — both AW and W must latch before memory update (order-independent)	DS §7 Commit	Critical	Directed (AW-first, W-first, concurrent)
FV-005	B channel response — BVALID / BRESP / BREADY timing for OKAY and SLVERR	DS §7 B	Critical	Directed + Scoreboard
FV-006	WSTRB byte-lane masking — only enabled bytes update memory; others preserved	DS §4 Write Path	High	Directed (all strobe patterns) + FC
FV-007	Out-of-range write — mem_w_en stays low; BRESP = SLVERR	DS §7 Commit	Critical	Directed + FC
FV-008	AR channel VALID/READY handshake — ARREADY gating by in-flight read	DS §7 AR	Critical	Directed + FC
FV-009	R channel response — RVALID / RDATA / RRESP timing and data correctness	DS §7 R	Critical	Scoreboard
FV-010	Out-of-range read — RDATA = 0, RRESP = SLVERR	DS §7 R	Critical	Directed + FC
FV-011	Write-before-read forwarding — same-cycle same-address WR/RD returnsWSTRB-merged data	DS §4 Forwarding	High	Directed + Scoreboard

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-01 2	Data integrity — every written value read back matches under all strobe patterns	DS §4	Critical	Constrained-Random + Scoreboard
FV-01 3	Backpressure tolerance — slave correctly stalls when master withholds BREADY / RREADY	DS §7	High	Directed + Constrained-Random
FV-01 4	Back-to-back transactions — no data loss when transactions issue every cycle	DS §7	High	Constrained-Random + Scoreboard
FV-01 5	Parameter scalability — DUV functions correctly at $\text{DATA_WIDTH} \in \{32, 64\}$ and varying MEM_DEPTH	DS §3 Parameters	Medium	Parameterized regression
FV-01 6	MEM_INIT preload — memory loads from hex file when MEM_INIT is non-empty	DS §5 Reset	Low	Directed
FV-01 7	AWPROT / ARPROT acceptance — slave accepts but does not reject any PROT encoding	DS §6 Ports	Low	Directed

Priority legend: Critical = must pass before MS sign-off; High = required for sign-off; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

The following functions are explicitly excluded from this plan because they will be verified at a different level of the verification hierarchy or are outside the scope of the AXI-Lite subset.

NC-ID	Function / Feature	Rationale for Exclusion	Verified In
NC-001	AXI4 burst transactions (INCR, WRAP, FIXED)	AXI-Lite restricts to single-beat transfers per the AMBA spec	N/A
NC-002	Exclusive access (AxLOCK, EXOKAY)	Not supported by AXI-Lite subset	N/A
NC-003	AxCACHE and AxQOS	Optional attributes not included in AXI-Lite core	N/A
NC-004	Decode errors (DECERR)	Address decode performed upstream in an interconnect, not by slave	Interconnect VP
NC-005	Clock domain crossing / metastability	Single-clock design; no asynchronous boundaries	N/A
NC-006	System-level bus contention and arbitration	Block-level VP; arbitration verified at sub-system level	Sub-system VP
NC-007	Gate-level timing and STA	Pre-silicon functional scope only; post-synthesis outside scope	STA flow
NC-008	Physical design, power, and DFT	Functional verification scope only	P&R / DFT flows

4. Verification Strategy and Methods

This section defines the overall strategy and the specific test and checking methods that the verification team will employ, consistent with Wilde Chapters 3–4 and 7–12. The strategy evolves across milestones from a simple directed testbench to a full UVM environment.

4.1 Overall Strategy

The verification effort is organized into five milestones that progressively increase rigor and automation. MS2 and MS3 are delivered together as a combined submission; the class-based environment, constrained-random stimulus, and functional coverage all come online in this submission.

Milestone	Methodology	Primary Goals	Status
MS1 – Traditional TB	Task-based module TB in SystemVerilog	Smoke-test basic read/write; confirm RTL compiles clean	✓ Done
MS2 – Class-Based TB	OOP SV classes (transaction, driver, monitor, scoreboard)	Layered TB; self-checking; scoreboard comparison	✓ Done (combined w/ MS3)
MS3 – Class-Based + CR	MS2 + constrained-random and functional coverage	Stimulus diversity; coverage-driven iteration	✓ Done (combined w/ MS2)
MS4 – UVM (Basic)	UVM 1.2 agent / env / sequencer / test	Full UVM structure; factory overrides; sequences	✓ Done
MS5 – UVM (Advanced)	MS4 + layered sequences, regression, coverage closure	Coverage closure; bug-injection validation; self-checking env; regression signoff	✓ Done

The overall strategy combines:

- Directed tests — to verify specific corner cases and compliance requirements.
- Constrained-random stimulus — to explore unanticipated scenarios and achieve broad coverage (MS3 onward).
- Functional coverage — measured via embedded covergroups in scoreboard (transaction-level) and monitor (cycle-level).
- Code coverage — to identify unreachable HDL constructs (line, branch, condition, toggle).
- Formal verification — applied selectively to FSM reachability and reset correctness (stretch goal).

4.2 Testbench Architecture

The MS1 testbench is a single-module, task-based testbench instantiating the DUT directly and driving all five AXI channels via `write_transaction` and `read_transaction` tasks. Starting at MS2, the testbench refactors into a class-based layered architecture. At MS4 the architecture

transitions to UVM 1.2 with a standard agent/env/test hierarchy; see §4.4 for the UVM architecture, hierarchy diagram, component descriptions, and messaging conventions.

4.2.1 Testbench Components

Functional coverage is embedded in existing components rather than implemented as a separate subscriber, following the pattern demonstrated by the course instructor. The scoreboard hosts transaction-level covergroups (sampled after each check); the monitor hosts cycle-level covergroups (sampled on every clock edge by a dedicated tracker thread).

Component	Description	Introduced In	Reuse / New
Top-level TB	Instantiates DUT, clock, interface bindings	MS1	New
AXI-Lite Interface	SV interface with master_cb / monitor_cb clocking	MS2	New
Transaction class	axi_lite_txn — addr, data, strb, mode (WR/RD/BOTH)	MS2	New
Generator	Random + directed transaction stream into mailbox	MS2	New
Driver	BFM driving AW/W/AR; reset; supports concurrent WR+RD	MS2	New
Monitor	Samples all channels; hosts cycle-level coverage	MS2	New
Scoreboard	Reference memory model; hosts txn-level coverage	MS2	New
Environment	Owns mailboxes; init/run/drain/stop/report lifecycle	MS2	New
Test library	test_base + 4 concrete tests	MS2	New
Functional coverage	11 covergroups (6 txn-level + 5 cycle-level)	MS2	New
UVM Agent	Packages driver + monitor + sequencer	MS4	New
UVM Env	Assembles agent, scoreboard, coverage	MS4	New
UVM Test(s)	Top-level tests extending uvm_test	MS4	New

4.3 Stimulus Methods

4.3.1 Directed Tests

Directed tests deterministically exercise specific DUT behaviors. The MS2 testbench implements four concrete tests, all extending a common test_base class:

Test Name	Purpose	Targets
test_random	20–50 random TXN_WRITE / TXN_READ transactions across the memory	FV-002/003, FV-005/006/009, FV-012, FV-014
test_wr_rd_same	Writes 0xDEAD_000N to 8 word-aligned addresses, reads each back	FV-005/009 OKAY, FV-014 WR-then-RD
test_byte_strobe	Writes 0xAABBCCDD with full strobe, then overwrites each byte lane individually; reads back 0x12345678	FV-006 strobe patterns
test_concurrent_rw	9 TXN_BOTH transactions exercising same-cycle AW/W/AR drive	FV-011 forwarding

The directed test matrix below catalogs additional DUT behaviors targeted across the milestones. Many will be reached by constrained-random stimulus introduced in MS3 rather than additional directed tests:

- Reset: assert / de-assert with in-flight transactions active
- Single write then single read at same address (data integrity)
- Walking-ones and walking-zeros data patterns
- All WSTRB patterns (0001 ... 1111 for DATA_WIDTH=32)
- In-range / boundary / out-of-range addresses (0x000, 0x3FC, 0x400)
- Back-to-back writes; back-to-back reads
- AW before W; W before AW; concurrent AW+W
- Slow BREADY / slow RREADY backpressure
- Write-before-read forwarding: WR and RD same cycle same address

4.3.2 Constrained-Random Tests

Implemented in MS2/MS3 (combined submission). The generator's run() task creates random transactions with the following constraints, enforced by transaction.sv and the generator's local rand_mem_addr() helper:

- c_addr_align: addr[1:0] == 2'b00 (32-bit word-aligned)
- c_wstrb_nonzero: WSTRB has at least one bit set on TXN_WRITE / TXN_BOTH
- c_mode_random: random transactions limited to TXN_WRITE / TXN_READ; TXN_BOTH is directed-only
- Address constraint by helper: addr held to in-range word window ($0 \leq \text{word_idx} < \text{MEM_DEPTH}$)
- Driver per-channel delay knobs: 0–N cycle random delays on AW, W, AR, BREADY, RREADY

Future closure work (in MS4/MS5) will extend constraints to bias under-covered bins (e.g., out-of-range addresses for SLVERR coverage, longer backpressure stalls).

4.3.3 Assertion-Based Verification

Not in scope for this revision.

4.3.4 Formal Verification

Stretch goal. Candidate formal targets:

- FSM reachability — all states of wr and rd submodules are reachable
- Deadlock freedom — no combination of backpressure can stall the slave permanently
- Reset completeness — every sequential element reaches its reset value in one clock

4.4 UVM Verification Plan

Milestone 4 rebuilds the verification environment on top of the Universal Verification Methodology (UVM 1.2). Each class-based MS2/MS3 component ports onto its UVM base class: the generator and mailbox pair is replaced by a `uvm_sequencer` driving a library of `uvm_sequence` classes; the driver, monitor, and scoreboard extend `uvm_driver`, `uvm_monitor`, and `uvm_scoreboard` respectively; and the env extends `uvm_env`. UVM-specific infrastructure replaces the hand-coded glue: `uvm_config_db` carries the virtual interface and configuration through the hierarchy, `uvm_phase` objections control simulation end, and TLM analysis ports replace the `mon2scb` mailboxes. The UVM factory (used through `type_id::create()`) enables future test variants via type overrides without modifying the environment code.

All UVM source files live in `AXI_Lite_MS4_UVM/tb/`. Every component and sequence is parameterized on `DATA_WIDTH`, `ADDR_WIDTH`, and `MEM_DEPTH` and registered with `uvm_object_param_utils` or `uvm_component_param_utils` accordingly. The testbench is launched by `axil_tb_top.run_test("axil_test")`, which instantiates the test, environment, agent, and all sub-components automatically through the UVM phasing mechanism.

4.4.1 UVM Architecture Overview

The MS4 architecture follows the standard UVM separation between stimulus, driver, monitor, and checker. Stimulus is generated by a library of `uvm_sequence` classes that emit sequence items into the sequencer; the driver pulls items from the sequencer and drives the DUT pins via the virtual interface; the monitor observes the same interface and publishes observed transactions on an analysis port; and the scoreboard subscribes to the analysis port and compares observed reads against a shadow memory model. Functional coverage remains embedded — six transaction-level covergroups inside the scoreboard and six cycle-level covergroups inside the monitor, all carried over unchanged from MS3.

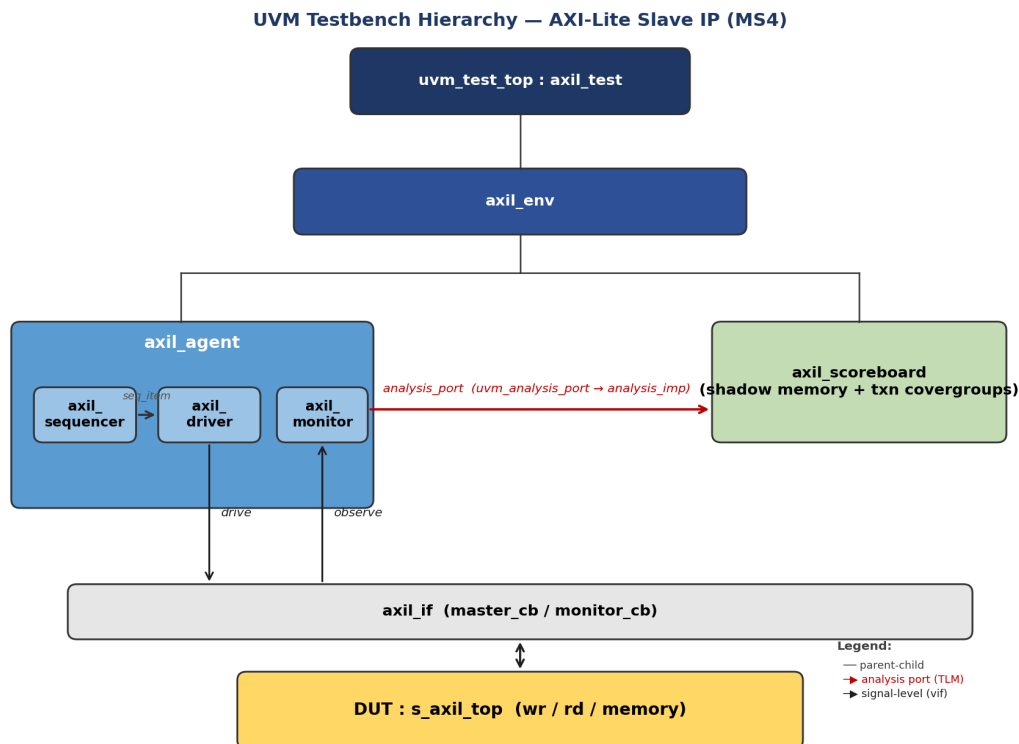
Key UVM features exercised in MS4:

- UVM phasing — `build_phase`, `connect_phase`, `end_of_elaboration_phase`, `run_phase`, and `report_phase` replace the MS3 `init()` / `run()` / `drain()` / `stop()` / `report()` lifecycle.
- Config DB — `uvm_config_db` propagates the virtual interface and the `MEM_INIT` file path from the top module down to every component that needs them.
- TLM analysis ports — `uvm_analysis_port` in the monitor publishes observed transactions; `uvm_analysis_imp` in the scoreboard receives them via a non-blocking `write()` callback.
- UVM factory — every component is constructed via `type_id::create()`, enabling test-level type overrides without touching the environment.
- Phase objections — `phase.raise_objection()` / `phase.drop_objection()` in the test's `run_phase` control simulation termination.

- Field automation macros — `uvm_field_*` macros on the sequence item enable automated print, copy, compare, pack, and record operations.

4.4.2 UVM Hierarchy

The diagram below shows the UVM component tree, the TLM analysis path from monitor to scoreboard, and the signal-level connections through the virtual interface.



The hierarchy is rooted at `uvm_test_top`, which is the `axil_test` instance created automatically by `run_test()`. The test owns one child, `env` (the `axil_env`). The environment in turn owns the `agent` and the `scoreboard`. The `agent` owns three children: the `sequencer`, the `driver`, and the `monitor`. The full UVM path of any component can be observed at run time via `uvm_top.print_topology()`, which is invoked in the test's `end_of_elaboration_phase`.

4.4.3 UVM Component Descriptions

The table below lists each UVM component and sequence class delivered in MS4, the base class it extends, and a brief description of its role. Detailed source is in the corresponding file under `AXI_Lite_MS4_UVM/tb/`.

Component	Extends	Brief Description
<code>axil_seq_item</code>	<code>uvm_sequence_item</code>	Transaction object carrying mode (WRITE/READ/BOTH), <code>addr</code> , <code>wdata</code> , <code>wstrb</code> , <code>prot</code> , <code>rdata</code> , <code>bresp</code> , <code>resp</code> . Field macros enabled for automated print/copy/compare. Three constraints: word-aligned address, non-zero <code>wstrb</code> on writes, and

Component	Extends	Brief Description
		mode restricted to WRITE/READ (BOTH unlocked in directed sequences).
axil_sequencer	uvm_sequencer	Parameterized typedef wrapper. Manages the flow of sequence items from running sequences to the driver via the standard seq_item_port / seq_item_export interface.
axil_driver	uvm_driver	Pulls items via seq_item_port.get_next_item(), drives the five AXI channels through the virtual interface (master_cb), then calls item_done(). All BFM tasks from MS3 (drive_aw, drive_w, drive_b, drive_read, drive_both) carry over with per-channel random-delay knobs.
axil_monitor	uvm_monitor	Passive observer. Five concurrent threads sample the bus: monitor_aw, monitor_w, merge_write, monitor_read, and cycle_coverage. Publishes merged write and read transactions on a single uvm_analysis_port. Hosts six cycle-level covergroups (cg_reset, cg_aw_w_hs, cg_ar_hs, cg_fwd, cg_bp_b, cg_bp_r).
axil_agent	uvm_agent	Active container for driver, sequencer, and monitor. Connects the driver's seq_item_port to the sequencer's seq_item_export and forwards the monitor's analysis port up to the environment for the scoreboard to subscribe to.
axil_scoreboard	uvm_scoreboard	Owns the shadow memory model (256x32). Receives transactions on an analysis_imp, checks writes (BRESP correctness, shadow update via WSTRB merge) and reads (RRESP correctness, RDATA vs shadow). Hosts six transaction-level covergroups (cg_bresp, cg_wstrb, cg_oor, cg_rresp, cg_addr, cg_b2b). Reports per-group and average coverage in report_phase.
axil_env	uvm_env	Top-level container. Instantiates the agent and scoreboard in build_phase; connects the agent's forwarded monitor port to the scoreboard's analysis_imp in connect_phase.
axil_test	uvm_test	Test root. Builds the environment, prints the UVM topology in end_of_elaboration_phase, raises an objection in run_phase, runs five sequences in turn (random, wr_rd, byte_strobe, concurrent, addr_oor), then drops the objection to end the simulation.
axil_base_seq	uvm_sequence	Common base for all sequences. Provides three protected helpers — send_write(), send_read(),

Component	Extends	Brief Description
		send_both() — built on start_item / finish_item with inline randomize-with constraints, plus a rand_mem_addr() helper for in-range word addresses.
5 derived seqs	axil_base_seq	axil_random_seq (50 CR transactions), axil_wr_rd_seq (write-then-read coherence), axil_byte_strobe_seq (WSTRB sweep), axil_concurrent_seq (TXN_BOTH), axil_addr_oor_seq (out-of-range SLVERR). Details in §4.4.4.

4.4.4 UVM Sequences

Stimulus is generated by a library of sequences derived from `axil_base_seq`, which extends `uvm_sequence #(axil_seq_item)` and provides three reusable helpers — `send_write()`, `send_read()`, and `send_both()` — using `start_item/finish_item` with inline randomize-with constraints. The test class instantiates each sequence and calls `.start(env.agent.sequencer)` from its `run_phase`.

Sequence	Type	Purpose
axil_random_seq	Constrained-random	Emits 50 word-aligned WRITE/READ transactions across the in-range memory window. Address constrained via the <code>rand_mem_addr()</code> helper to $0 \leq \text{word_idx} < \text{MEM_DEPTH}$.
axil_wr_rd_seq	Directed	Writes the pattern 0xDEAD_000N to eight word-aligned addresses then reads each back to confirm data integrity.
axil_byte_strobe_seq	Directed	Builds the value 0x12345678 one byte at a time using single-byte WSTRBs over the same address, plus a sweep across all non-zero strobe values (1..15) to exercise WSTRB byte-lane masking.
axil_concurrent_seq	Directed	Issues TXN_BOTH transactions that drive AW + W + AR simultaneously on the same clock edge. Exercises the DUT's write-before-read forwarding path (FV-011).
axil_addr_oor_seq	Directed	Drives a write and a read at $\text{MEM_DEPTH} * (\text{DATA_WIDTH}/8) + 4$ to confirm <code>BRESP/RRESP = SLVERR</code> on out-of-range accesses (FV-007 / FV-010).

4.4.5 UVM Messaging and Logging

All testbench prints route through the UVM reporting subsystem rather than `$display`. Components use the four standard severity macros with consistent ID strings so output can be filtered at run time. Verbosity levels are graded so the default run produces a readable transcript while a higher verbosity reveals every component-creation and per-transaction event.

Macro	Severity	Used For
<code>uvm_info</code>	INFO	Per-transaction prints, lifecycle events, end-of-test summaries, coverage reports.
<code>uvm_warning</code>	WARNING	Recoverable anomalies (e.g., shadow-memory write outside MEM_DEPTH).
<code>uvm_error</code>	ERROR	Scoreboard mismatches, unexpected BRESP/RRESP encodings. Counted toward the final pass/fail decision.
<code>uvm_fatal</code>	FATAL	Configuration failures that prevent the testbench from running (e.g., virtual interface not found in <code>uvm_config_db</code>). Terminates simulation immediately.

The ID-string convention identifies the originating component at a glance and allows targeted filtering with `+uvm_set_action` and `+uvm_set_severity` plusargs:

ID String	Originating Component / Use
DRV	Driver — <code>drive_write</code> , <code>drive_read</code> , <code>drive_both</code> , per-channel handshake events
MON	Monitor — observed transactions, merged write pairs, end-of-test cycle-level coverage report
SCB	Scoreboard — per-transaction check results, error reports, end-of-test summary and txn-level coverage report
ENV	Environment construction events
AGT	Agent construction events
SEQ	Sequencer / sequence body progress and completion messages
TEST	Top-level test scenario transitions (start/end of each sequence)
CFG	Configuration database failures (fatal)

Verbosity defaults to `UVM_MEDIUM`. End-of-test summaries (pass/fail counts, per-covergroup percentages, RESULT line) are emitted at `UVM_NONE` so they always appear in the transcript regardless of the verbosity setting. Per-transaction driver and monitor prints are at `UVM_MEDIUM`. Component-creation traces are at `UVM_HIGH` for deep debugging. Runtime control is via `+UVM_VERBOSITY=UVM_HIGH` on the `simv` command line, or per-component overrides via `+uvm_set_verbosity=axil_env.agent.mon,_ALL_,UVM_HIGH,run`.

4.4.6 UVM Factory and Field Macros

Every UVM component and sequence item is registered with the UVM factory via `uvm_component_param_utils` (parameterized components) or

`uvm_object_param_utils` (sequence items and sequences). Components are constructed with `type_id::create("name", parent)` rather than `new()` directly; this allows tests to substitute a derived class via `set_type_override_by_type()` without changing the environment code. This pattern is reserved for MS5 negative tests and alternate driver behavior, but the infrastructure is already in place from MS4.

Field macros are enabled on `axil_seq_item` using the `uvm_field_*` family inside the `uvm_object_param_utils_begin/uvm_object_utils_end` block. The macros automate `print()`, `copy()`, `compare()`, `pack()`, and `record()` based on the registered fields, eliminating boilerplate code on every transaction class:

```
`uvm_object_param_utils_begin(axil_seq_item #(DATA_WIDTH, ADDR_WIDTH))
  `uvm_field_enum(txn_mode_e, mode, UVM_ALL_ON)
  `uvm_field_int(addr, UVM_ALL_ON | UVM_HEX)
  `uvm_field_int(wdata, UVM_ALL_ON | UVM_HEX)
  `uvm_field_int(wstrb, UVM_ALL_ON | UVM_BIN)
  `uvm_field_int(prot, UVM_ALL_ON | UVM_BIN)
  `uvm_field_int(rdata, UVM_ALL_ON | UVM_HEX)
  `uvm_field_int(bresp, UVM_ALL_ON | UVM_BIN)
  `uvm_field_int(rresp, UVM_ALL_ON | UVM_BIN)
`uvm_object_utils_end
```

The `UVM_ALL_ON` flag enables every automated operation; the `UVM_HEX` and `UVM_BIN` flags control the format used when `print()` or waveform recording renders each field. A custom `convert2string()` method overrides the auto-generated print formatting where a compact one-line representation is preferred for transaction-level log messages emitted by the driver, monitor, and scoreboard.

4.5 Checking Strategy

Result checking uses the following methods:

- Scoreboard-based checking — TB maintains a reference memory model; every read response is compared against the expected value computed from the sequence of writes (with WSTRB merging).
- Response-code checking — every BRESP and RRESP is compared against the expected OKAY/SLVERR based on address range.
- End-of-simulation report — summary of pass/fail counts and coverage percentages.

5. Verification Environment

5.1 Simulation Environment

Field	Value
Simulator	Synopsys VCS
Simulator Version	VCS R-2020.12 (PSU ECE server installation)
Verification Methodology	Traditional (MS1) → Class-based (MS2-3) → UVM 1.2 (MS4-5)
HDL Language	SystemVerilog IEEE 1800-2017
Compile Options	-sverilog -ntb_opts uvm-1.2 -timescale=1ns/1ps +define+UVM_NO_DEPRECATED -full64
Coverage Options	-cm line+cond+fsm+tgl+branch+assert -cm_dir ./cov.vdb
Sim / Runtime Options	+vcs+lic+wait +fsdb+autoflush -assert svaext
Waveform Format	FSDB (Verdi) — \$fsdbDumpvars already present in TB
Regression Framework	Makefile-driven; targets: compile, sim, cov, verdi, clean

5.2 Coverage Plan

Functional coverage groups map directly to the FV-IDs in Section 2. Coverage is embedded in the testbench components: transaction-level covergroups live in the scoreboard (sampled after each scoreboard check), and cycle-level covergroups live in the monitor (sampled on every clock edge by a dedicated tracker thread). The ‘Achieved’ column shows the final coverage closed at MS5 across a 50-seed constrained-random regression plus directed sequences; the ‘Target’ column shows the closure goal for MS5 sign-off. All eleven functional covergroups reached 100%.

FV-ID(s)	Coverage Group	Owner	Bins / Cross Points	Achieved	Target
FV-001	cg_reset	Monitor	Reset asserted / released states	100%	100%
FV-002/003	cg_aw_ws	Monitor	AW/W handshake order: AW-first, W-first, concurrent	100%	100%
FV-005	cg_bresp	Scoreboard	BRESP $\in$ {OKAY, SLVERR} $\times$ BREADY delay bins	100%	100%
FV-006	cg_wstrb	Scoreboard	WSTRB values 0001 ... 1111 (15 non-zero patterns)	100%	100%
FV-007/010	cg_oor	Scoreboard	Out-of-range addr $\times$ op {WR, RD} $\times$ response	100%	100%

FV-ID(s)	Coverage Group	Owner	Bins / Cross Points	Achieved	Target
FV-008	cg_ar_hs	Monitor	AR-to-R latency bins {1, 2, 3, 4-7, 8-15, 16+}	100%	100%
FV-009	cg_rresp	Scoreboard	RRESP $\in$ {OKAY, SLVERR}	100%	100%
FV-011	cg_fwd	Monitor	WR + RD same cycle $\times$ address match (forwarding)	100%	100%
FV-012	cg_addr	Scoreboard	Word addresses binned across [0, MEM_DEPTH-1]	100%	90%
FV-013	cg_bp_b/cg_bp_r	Monitor	BREADY/RREADY stall bins {0, 1-3, 4-7, 8-15, 16+}	100%/100%	90%
FV-014	cg_b2b	Scoreboard	Back-to-back WR-WR, RD-RD, WR-RD, RD-WR pairs	100%	100%

MS5 Code Coverage Results: A 50-seed constrained-random regression, augmented with directed sequences for unaligned and out-of-range addresses and a dedicated no-initialization build, closed code coverage to an overall score of 96.77%. Line coverage, branch coverage, and functional coverage all reached 100%; toggle coverage reached 98.55% and condition coverage reached 85.29%. FSM coverage is reported as not-applicable because the control logic is implemented with independent handshake flag registers (aw_active, w_active, b_pending, ar_active, read_pending, r_valid) rather than an encoded finite state machine, so no FSM shape exists for the tool to measure. All 50 seeds passed with zero scoreboard errors and zero UVM_ERROR/UVM_FATAL.

5.3 Unreachable Coverage Analysis

The residual condition, toggle, and branch coverage gaps remaining after MS5 closure were individually reviewed and determined to be architecturally unreachable. They are documented here as approved waivers rather than open holes.

BRESP and RRESP bit 0 toggle (affected modules: s_axil_top, s_axil_wr, s_axil_rd, axil_if). The slave only generates two AXI-Lite response codes: RESP_OKAY (2'b00) and RESP_SLVERR (2'b10). Bit 0 is permanently 0 in both encodings. The codes RESP_EXOKAY (2'b01) and RESP_DECERR (2'b11) would require bit 0 to be 1, but this slave never produces them. This is a structural property of the RTL, not a stimulus gap.

MEM_INIT == "" branch (s_axil_top, lines 73 and 80). The testbench always passes a non-empty MEM_INIT parameter when instantiating the DUT. The else-branch (empty string, skip memory load) would only execute if the design were instantiated without a memory initialisation file. Because the parameter is hard-coded as non-empty in the testbench top level, this path is structurally unreachable in the current environment.

VALID-while-not-READY handshake conditions (s_axil_wr lines 85 and 105; s_axil_rd line 77). The slave asserts awready, wready, and arready purely combinatorially as a function of their respective active flags. The UVM driver follows a ready-before-valid ordering, waiting for the slave to become ready before asserting valid. By the time valid rises, ready is already high, so the condition valid=1 && ready=0 is never seen on any of these channels.

BREADY-before-BVALID and back-to-back write-while-response-pending conditions (s_axil_wr lines 141 and 118). The UVM driver asserts s_axi_bready only after issuing a write transaction,

in direct response to `b_pending` going high; it never pre-asserts `bready` speculatively, so `b_pending=0 && bready=1` is never exercised. Similarly, the driver enforces a strict request-response sequence and waits for the B-channel handshake to complete before issuing the next write, so the condition `aw_active && w_active && b_pending` (`write_fire` blocked) is never produced.

5.4 Bug Injection and Environment Validation

To demonstrate that the verification environment actively detects failures rather than passively passing, two bugs were injected — one in the RTL and one in the environment — and the environment was shown to catch each. Both were reverted after demonstration; the design and environment under submission are clean.

RTL bug — WSTRB byte-lane masking defeated. The byte-strobe assignment in `s_axil_wr` was mutated so the memory write ignores WSTRB and unconditionally writes all four byte lanes. The scoreboard's data-path checker detected the resulting corruption on read-back, reporting 261 READ MISMATCH errors across the 50-seed run; the mismatch signature (expected values carrying preserved/zero byte lanes against a DUT that wrote all lanes) directly identifies the defect as a byte-enable failure. This validates FV-006 (WSTRB masking) and FV-012 (data integrity) checking.

Environment bug — monitor-to-scoreboard analysis port disconnected. The analysis-port connection between the monitor and the scoreboard in the environment `connect_phase` was commented out, simulating a refactoring error that severs the checker from observed traffic. Before this milestone the scoreboard would have reported PASS in this condition because its error counter remained zero — a silent false-positive. A new scoreboard `check_phase` was therefore added that asserts a non-zero observed-transaction count at end of test; with the port disconnected it fires UVM_ERROR ("ENV CHECK FAILED: scoreboard observed ZERO transactions"), the dedicated `errors.log` captures it, and the scoreboard reports FAIL. This demonstrates the environment is self-checking against its own mis-configuration.

5.5 MS5 Environment Enhancements

Self-checking scoreboard (`check_phase`). The scoreboard now verifies at end of test that it observed a non-zero number of transactions and reports a consistent PASS/FAIL result that accounts for both data-mismatch errors and the zero-transaction structural condition described in §5.5.

Multi-file logging. In addition to the master UVM transcript (`axil_uvm.log`), per-component logs are produced by routing report IDs to separate files: `driver.log` (DRV), `monitor.log` (MON), and `scoreboard.log` (SCB). A dedicated `errors.log` captures every UVM_ERROR and UVM_FATAL through a `uvm_report_catcher`, independent of UVM file-routing precedence, so errors are never lost behind per-ID routing. On a passing run `errors.log` is empty.

Startup banner. The test prints a startup banner at `start_of_simulation` identifying the project, team, test name, and time, providing an at-a-glance header in every transcript.

6. Required Tools

This section lists all software tools required to support the described verification environment, consistent with Wilde Chapters 5–6.

Category	Tool / Package	Version	Vendor / Source	Status
Simulator	Synopsys VCS	R-2020.1 2	Synopsys	[X] Licensed (PSU lab)
Coverage / URG	VCS Coverage + URG	R-2020.1 2	Synopsys	[X] Bundled with VCS
Waveform Viewer	Synopsys Verdi (FSDB)	R-2020.1 2	Synopsys	[X] Licensed
Linting	Spyglass Lint (optional)	—	Synopsys	[] Optional
Formal Tool	Not planned for MS1–MS5	—	—	[] N/A
VIP	None (hand-coded AXI-Lite driver)	—	Internal	[X] In-house
Regression	GNU Make	≥ 4.2	GNU	[X] Available
Version Control	Git / GitHub	≥ 2.30	GitHub	[X] Available
Bug Tracking	GitHub Issues	—	GitHub	[X] Available
Documentation	MS Word / PDF / Markdown	—	—	[X] Available

7. Completion Criteria

Verification is declared complete when ALL of the following measurable criteria have been satisfied. Each criterion must be signed off by the team before submission of each milestone.

7.1 Simulation Completion Criteria

#	Criterion	Target Threshold	Applies To
1	All directed tests pass with zero unexpected failures	100% pass rate	MS1–MS5
2	Constrained-random regression passes across N seeds	$N \geq 50$ seeds	MS3–MS5
3	All functional coverage groups close to specified targets	$\geq 90\%$ avg, 100% critical	MS3–MS5
4	Statement and branch code coverage achieved	$\geq 90\%$	MS2–MS5
5	Toggle coverage achieved on DUT outputs and interfaces	$\geq 85\%$	MS3–MS5
6	FSM state and transition coverage achieved	100% reachable	MS3–MS5
7	All open bugs triaged; no Critical or High severity bugs outstanding	0 Critical / High open	All MS
8	Waived coverage items documented and approved	Approval on record	MS3–MS5

7.2 Formal Verification Completion Criteria

Formal verification is a stretch goal (not required for MS1–MS5). Criteria apply only if formal is pursued.

#	Criterion	Target	Status
1	All formal properties proven or bounded-proven	PROVEN / BOUNDED	[] Stretch goal
2	No vacuous passes (vacuity checks passed)	0 vacuous	[] Stretch goal
3	All counterexamples analyzed and resolved	0 open CEX	[] Stretch goal

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
MS1 directed TB runs clean on VCS	Patrick, Pratibha	04/20/2026	✓ Done
MS2/MS3 class-based TB + CR + coverage (combined submission)	Patrick, Pratibha	05/07/2026	✓ Done

Sign-Off Item	Responsible Party	Date	Sign
MS4 UVM env runs base test clean	Patrick, Pratibha	05/14/2026	✓ Done
MS5 coverage closure + bug injection + regression signed off	Patrick, Pratibha	05/28/2026	✓ Done
All high-priority bugs closed or deferred	Patrick, Pratibha	05/28/2026	✓ Done
Final zip submitted to Canvas	Patrick, Pratibha	05/28/2026	✓ Done

8. Resources and Schedule

8.1 Human Resources

The team consists of two engineers sharing all responsibilities. For each milestone, one team member leads RTL/design tasks while the other leads verification/testbench tasks; roles alternate across milestones to ensure both members gain full exposure to every aspect of the project.

Role	Name(s)	Allocation	Responsibilities
Project Lead (shared)	Patrick, Pratibha	0.5 FTE each	Planning, schedule tracking, Canvas submission, instructor interface
Design / RTL	Patrick, Pratibha	shared	RTL maintenance, bug fixes, parameter extensions
Verification Lead	Patrick, Pratibha	shared	TB architecture, scoreboard, coverage, UVM transition
Documentation	Patrick, Pratibha	shared	HLDS, VP, milestone READMEs, final report

Per-Milestone Task Split

Task ownership rotates each milestone so both members cover the full workflow. Shared tasks (review, debug, integration) are owned jointly.

Milestone	RTL / Design Owner	Verification / TB Owner	Shared
MS1	Patrick (initial RTL, defs)	Pratibha (trad TB, tasks)	Git setup, directory structure, review
MS2	Pratibha (RTL tweaks, SV constructs)	Patrick (class-based TB, scoreboard)	Debug, integration
MS3	Patrick (parameter extension, coverage RTL)	Pratibha (CR tests, covergroups)	Coverage closure
MS4	Pratibha (RTL freeze, UVM bind)	Patrick (UVM agent, env, base test)	UVM bring-up debug
MS5	Patrick (coverage closure, regression)	Pratibha (layered seqs, bug injection, env enhancements)	Coverage closure, final report
FINAL	Both — consolidation	Both — final regression & packaging	Canvas submission

8.2 Hardware Resources

Resource	Specification	Quantity	Availability
Simulation Compute	PSU ECE lab workstations or personal laptops (≥ 8 GB RAM)	2	[X] Available
Shared Storage	GitHub repository for all code, docs, and transcripts	1 repo	[X] Available
Local Disk	≥ 5 GB per milestone workspace for waveforms/coverage DBs	as needed	[X] Available
Emulation / FPGA	Not required for this project	—	[] N/A

8.3 Schedule

MS2 and MS3 are delivered together as a combined submission. Each subsequent milestone begins the day after the previous milestone's sign-off.

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
M1	MS1 – Design Spec, VP, RTL, traditional TB	04/14/2026	04/20/2026	Both	[X] Done
M2/ M3	MS2/MS3 – Class-based TB + CR + functional coverage	04/21/2026	05/07/2026	Both	[X] Done
M4	MS4 – UVM environment (basic)	05/08/2026	05/14/2026	Both	[X] Done
M5	MS5 – Coverage closure + bug injection	05/15/2026	05/24/2026	Both	[X] Done
M6	Final regression run across 50+ seeds	05/25/2026	05/26/2026	Both	[X] Done
M7	Final report + waveform snapshots	05/26/2026	05/27/2026	Both	[X] Done
M8	Canvas submission (ece592s26_<team>_final_project.zip)	05/28/2026	05/28/2026	Both	[X] Done

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R 1	RTL bugs in MS1 baseline (write latch clear, \$urandom seed) delay MS2 scoreboard bring-up	Closed	—	Resolved as of MS2: all four directed tests (test_random, test_wr_rd_same, test_byte_strobe, test_concurrent_rw) pass with 0 scoreboard errors. The MS1 testbench bugs were fixed by the teammate. Latent RTL latch-clear concern did not manifest in MS2 random + directed runs
R 2	Coverage closure not achieved within schedule	Medium	High	Add coverage-driven constrained-random tests; schedule coverage reviews at mid-MS3 and mid-MS5; waiver process for unreachable bins
R 3	VCS license or lab access unavailability	Low	High	Both team members use PSU remote lab / ECE server SSH; schedule long coverage regressions off-peak to avoid license contention
R 4	Team member unavailable (illness, conflict)	Medium	Medium	Both members cross-trained; all commits reviewed by the other; documentation kept current; tasks rotate each MS
R 5	UVM bring-up (MS4) slips due to methodology learning curve	Medium	Medium	Start UVM skeleton in parallel with MS3 CR work; reuse class-based TB driver/monitor logic inside UVM components
R 6	Parameter extension (DATA_WIDTH=64) reveals latent RTL bugs	Low	Medium	Limit FV-015 to MS3 regression; raise as separate bugs rather than blocking sign-off if issues found
R 7	Scope creep from optional features (formal, MEM_INIT, AxPROT semantics)	Medium	Low	Keep optional items explicitly marked Low priority in Section 2; defer to FINAL milestone if time permits
R 8	Git merge conflicts between team members	Medium	Low	Use feature branches; one PR reviewer required before merge; separate directories per MS reduces overlap

10. Document Revision History

Revision	Date	Author	Description of Change	Approved By
0.1	2026-04-19	Patrick, Pratibha	Initial draft for internal review	—
1.0	2026-04-20	Patrick, Pratibha	Baselined for MS1 submission	—
1.1	2026-04-30	Patrick, Pratibha	MS2/MS3 combined submission updates: §4.1 strategy table now shows MS2/MS3 delivered together; §4.3.1 lists 4 concrete tests (test_random, test_wr_rd_same, test_byte_strobe, test_concurrent_rw); §4.3.2 reflects implemented constraints; §4.3.3 SVA marked out of scope; §5.2 adds Achieved column with measured coverage; §7.3 sign-off and §8.3 schedule combined for MS2/MS3; R1 closed (zero scoreboard errors observed)	—
1.2	2026-05-14	Patrick, Pratibha	MS4 UVM submission: new §4.4 UVM Verification Plan added (architecture overview, hierarchy diagram, component descriptions, sequence library, messaging/logging conventions, factory and field macros); §4.2 forward reference updated to point at §4.4; §4.4 Checking Strategy renumbered to §4.5; §7.3 MS4 sign-off marked Done; §8.3 M4 schedule row marked Done; document revision bumped to 1.2.	—
1.3	2026-05-28	Patrick, Pratibha	MS5 final submission: §5.2 coverage table updated to final achieved values (all 11 covergroups 100%); new §5.3 unreachable coverage analysis; §5.4 bug injection and environment validation (RTL WSTRB bug + env disconnect bug); §5.5 environment enhancements (check_phase, multi-file logging, banner); §4.1 MS4/MS5 strategy rows marked Done; simulator version corrected to	—

Revision	Date	Author	Description of Change	Approved By
			VCS R-2020.12; §7.3 MS5 sign-off and §8.3 M5–M8 marked Done.	

End of Verification Plan — AXI-Lite Slave IP with Memory — VPL-AXIL-001